

LOADING...



Spring MVC에서 RestClient → WebClient 전환으로 TPS 94배 향상하기

우아한테크코스 BE 7기 메이



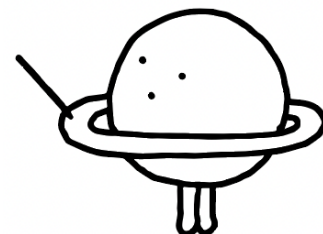
이 발표의 타겟층

외부 API 호출에 별 생각 없이 RestClient를 사용하는 사람

운영하는 서비스에 외부 API 호출이 많아서 지연 시간이 고민인 사람

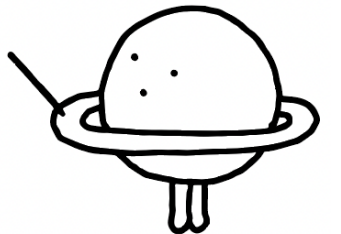
RestClient와 WebClient의 차이를 정확히 알지 못하는 사람

WebClient가 왜 논블로킹이라고 하는지 알고 싶은 사람

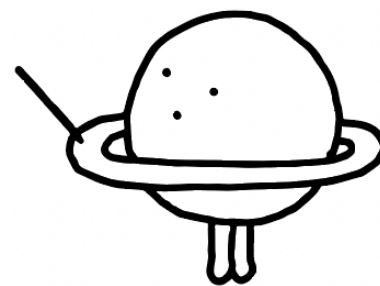


목차

1. 기능 설명
2. 기존 구현과 문제
3. 테스트 설계와 기존 구현 테스트(Test1)
4. WebClient 전환(Test2)
5. 톰캣 스레드 블로킹 문제 개선하기(Test3)
6. 결과 비교, 결론



기능 설명





랜덤 여행지 - 연관 관광지 조회 기능



랜덤 여행

인천 여행 영상 1개

투립 지역 카테고리(서울, 인천, 강릉, 속초 ...) 중 랜덤 여행지 지정



인천 영종도 2박 3일 여행 서울 근교...

%S · %S



영종도

인천

1박 2일

10개 장소

인천 연관 관광지

관광지

구읍벤티

월미도 · 을왕리해수욕장 ·
인천차이나타운 · 강화루지 · 송도달...

8곳 >

쇼핑

송도 트리플스트리트

와이케이 쇼핑아울렛

2곳 >

음식점

서울호떡 영종도점

스시아인앤 영종도점 · 엄마손집밥 ·
서해안조개광장 · 고집132 본점 · 고...

6곳 >

카페

카페 얼트

자연도소금빵 본점 · 빵드오늘
하늘도시점 · 토모루베이커리 영종하...

7곳 >

영상을 선택하면 여행을 시작할 수 있어요

다시 돌리기

이 여행 시작하기



한국관광공사
KOREA TOURISM ORGANIZATION

연관 관광지 정보 제공



투립과 한국관광공사에서 다루는 지역 차이



시/도 단위(광역자치단체)
+
시/군/구 단위(기초자치단체)
Ex) 서울, 부산, 강릉, 속초, ...

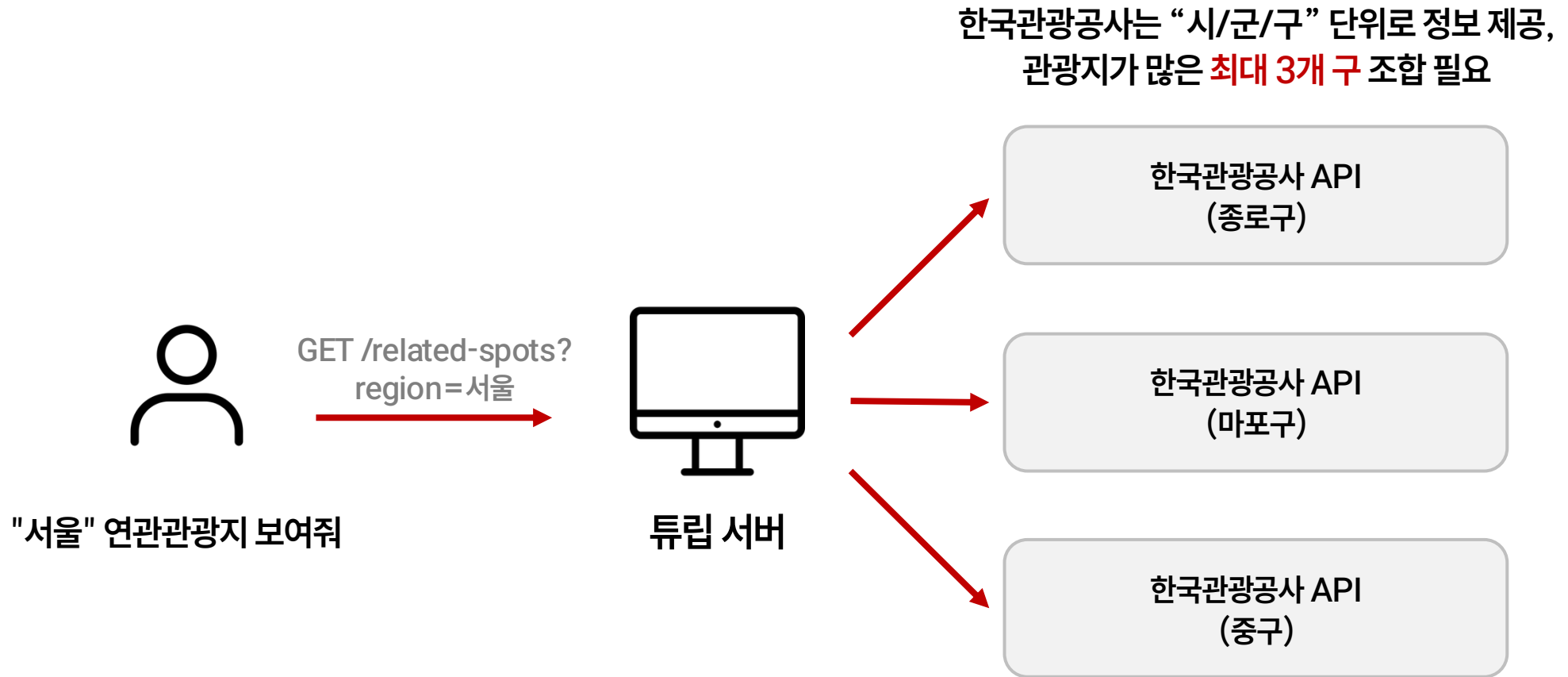
불일치



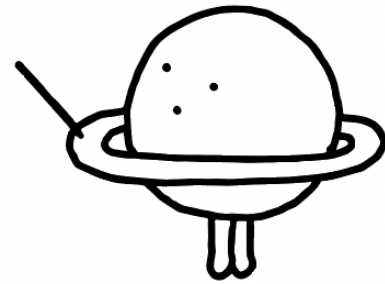
한국관광공사
KOREA TOURISM ORGANIZATION

시/군/구 단위(기초자치단체)
Ex) 종로구, 중구, 마포구, ...

연관관광지 조회 시 동작

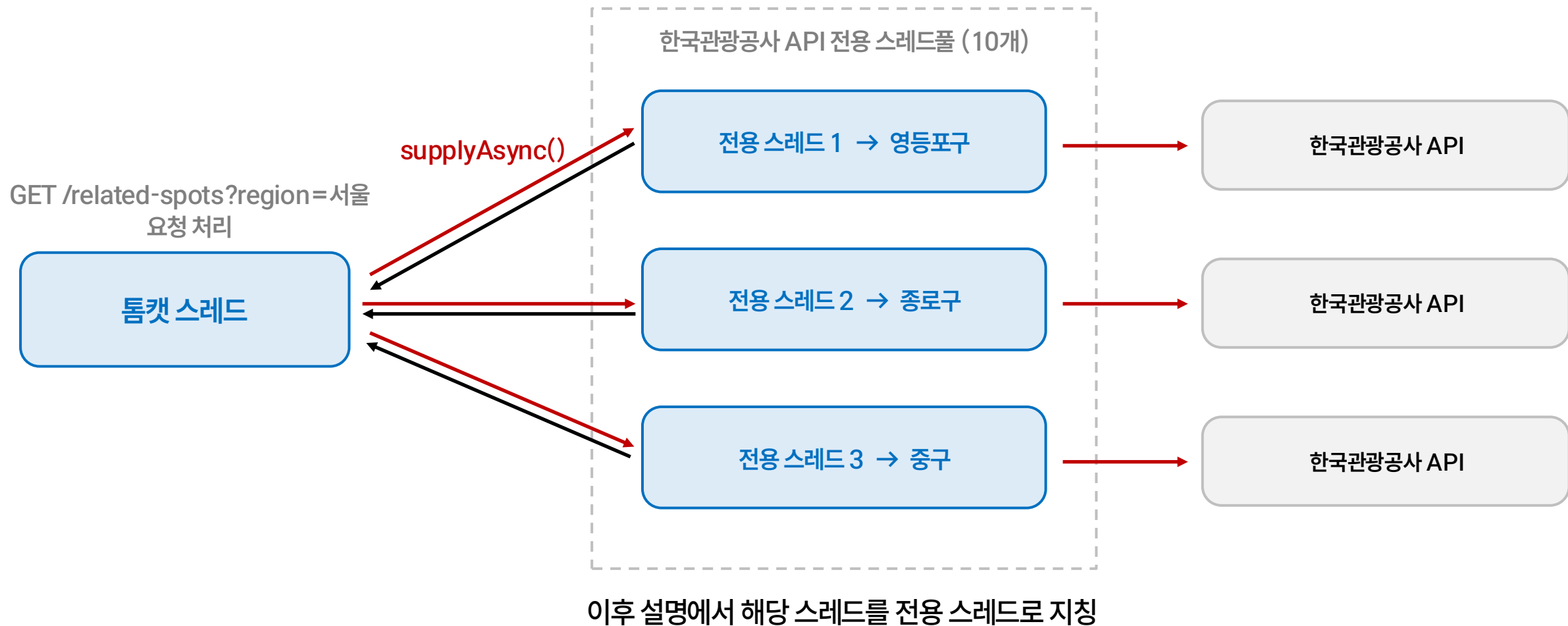


기존 구현과 문제

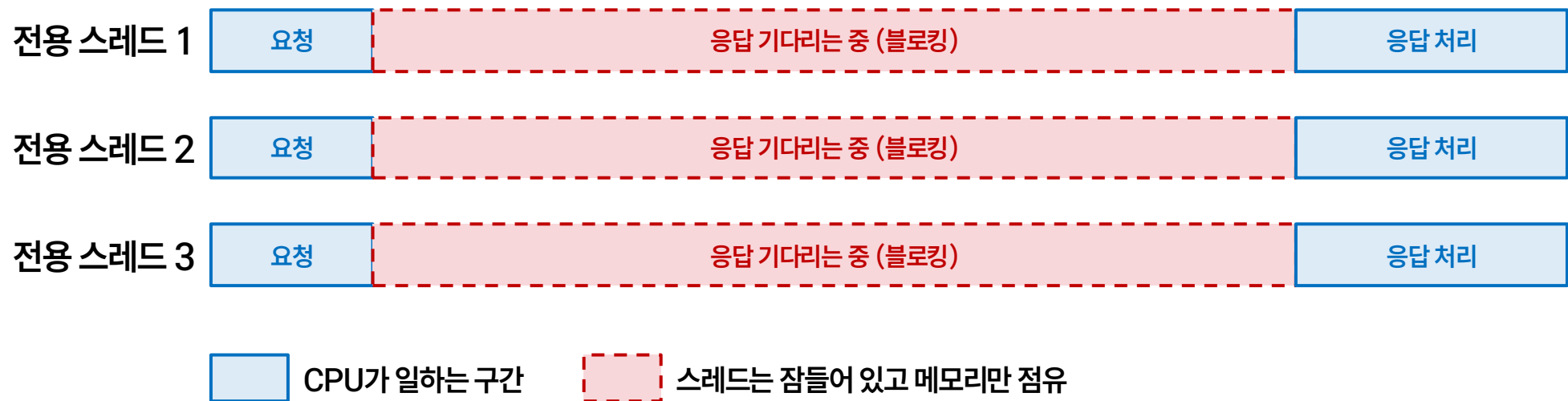




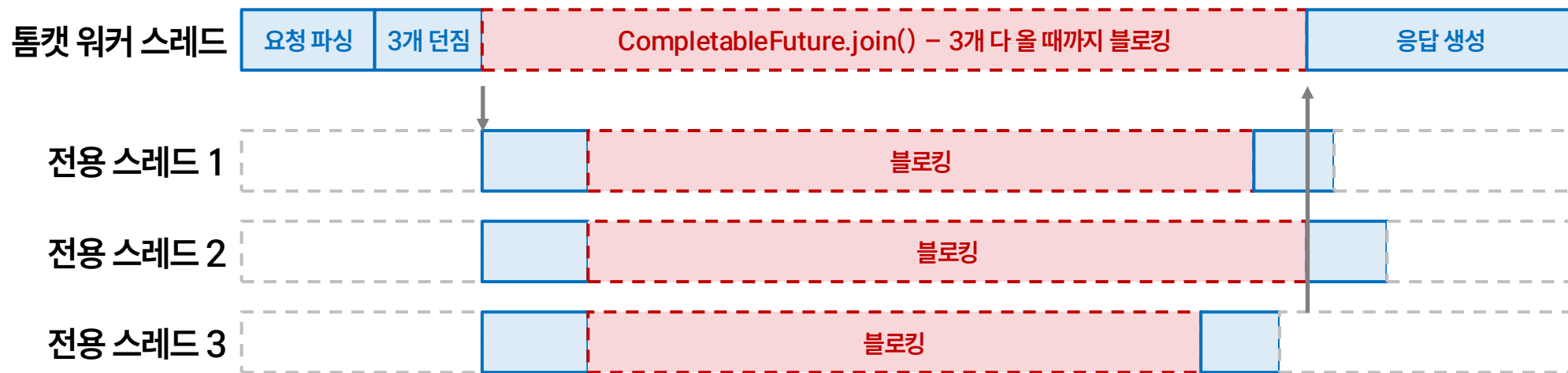
기존 구현 - RestClient + CompletableFuture 기반 비동기 병렬 호출



문제 1) 외부 서버를 찌르는 전용 스레드는 블로킹된다



문제 2) 비동기로 던져도 join()에서 톰캣 스레드가 블로킹된다



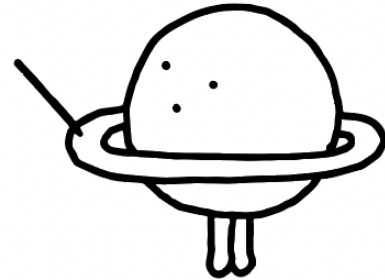
요청 1개당 스레드 4개가 대부분의 시간을 기다리기만 한다

- 비동기 병렬 호출: 스레드 20개가 걱정이면 WebClient(비동기 논블로킹)로 바꾸면 스레드 점유 자체를 줄일 수 있음. 지금 RestTemplate 같은 블로킹 방식이면 5개 호출 = 스레드 5개 잠깐 점유되는 거라 실제로 큰 부담은 아닐 수도 있어요 (요청당 처리시간이 짧다면)
- 지금 사용하는 RestClient 쓰면 블로킹됨
 - WebClient 써야 논블로킹 호출 가능
- 일단 RestClient로 구현해봤다가 → 부하테스트 하고 → WebClient로 바꿔봐도 좋을듯

블로킹이 문제인거면 논블로킹이라고 하는 WebClient를 사용하면 되지 않을까?



테스트 설계와 기존 구현 테스트(Test1)



테스트 규칙 - 변수를 최대한 통제하기



서버 재시작

애플리케이션 캐싱 영향 제거 — CD Action 재실행 후 top으로 CPU 1% 이하 확인



k6 인스턴스 CPU 확인

요청 수 변수 제거 — t4g.micro 스로틀링(약 20%)에 안 걸리는지 매 테스트 확인



부하테스트 DB 재시작

버퍼풀 캐싱 영향 제거 — 이 API는 DB를 찌르진 않지만 동일 조건 유지



동일한 워밍업 쿼리셋

DB EC2의 OS 페이지 캐시를 매 테스트 같은 상태로

결과 차이가 캐시 때문이 아니라 구현 차이 때문임을 보장

가설 1

저부하: WebClient로 바뀌도 latency는 별 차이 없다
고부하(동시 요청 > 전용 스레드풀 10개): WebClient가 더 낮고 안정적이다

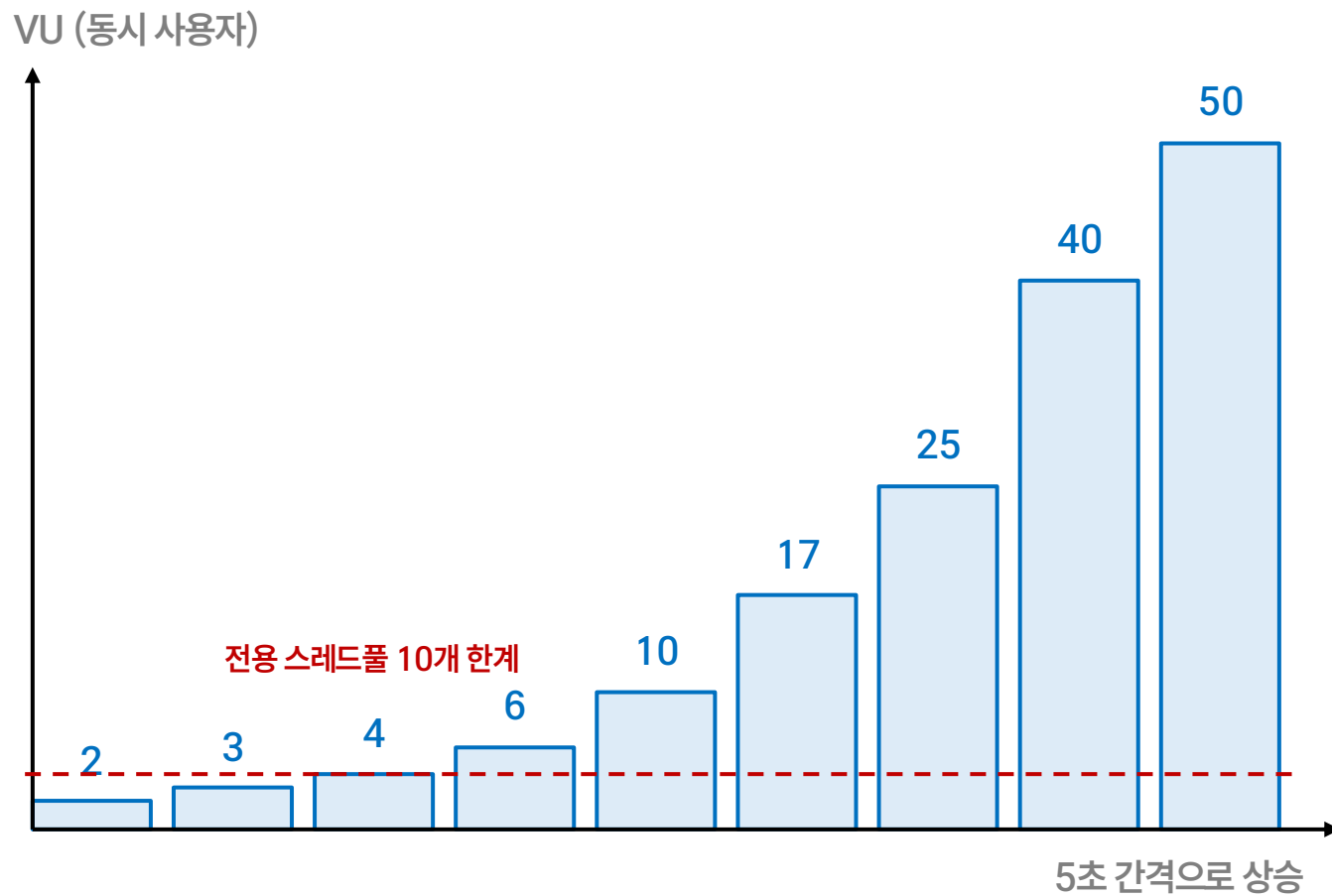
가설 2

WebClient로 전환하면 처리량(TPS)이 늘어난다

가설 3

Controller에서 Mono를 return하면 처리량이 더 늘어난다

테스트 시나리오

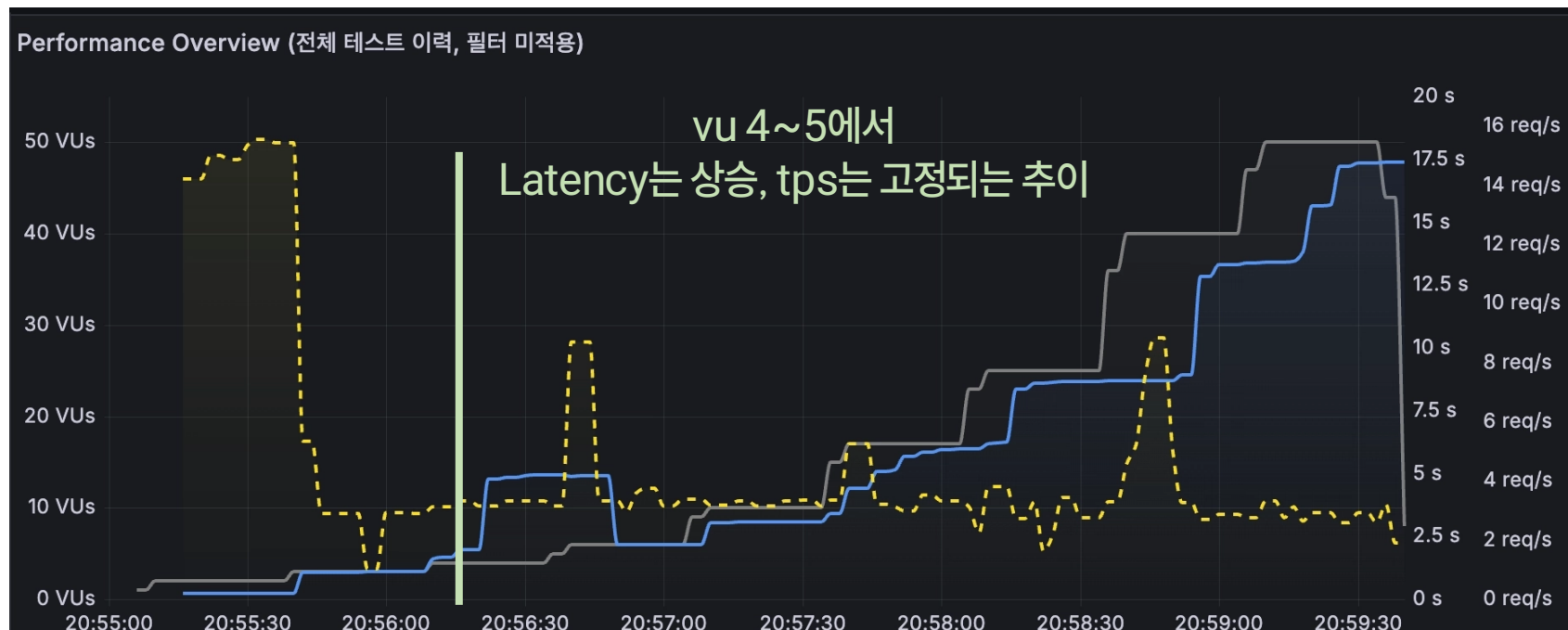


사용자는 서울 · 인천 · 부산(3개 시/군/구 조합하는 지역) 중 하나의 연관관광지 조회 API를 호출

실제 한국관광공사 API를 호출
(모의 서버 X)

vu 2 → 50까지 계단식으로 부하를 올리며 latency, TPS, 스레드 상태를 관찰

Test 1 결과 분석



응답 시간(latency) 처리량(tps)

3.07

평균 TPS

3.33

최대 TPS

13.2 s

p95 latency

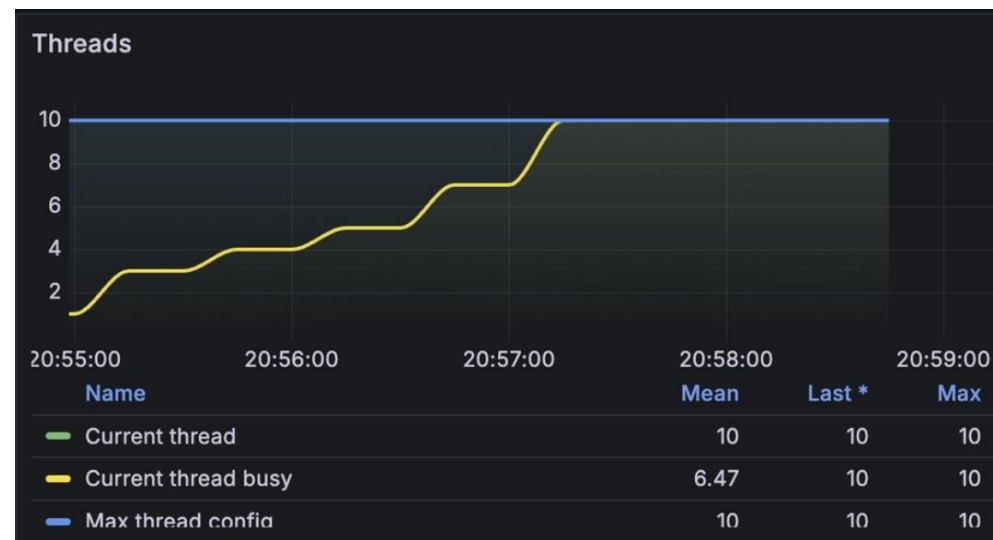
16.8 s

p99 latency

전용 스레드가 10개뿐이라 조금만 몰려도 대기시간이 길어진다

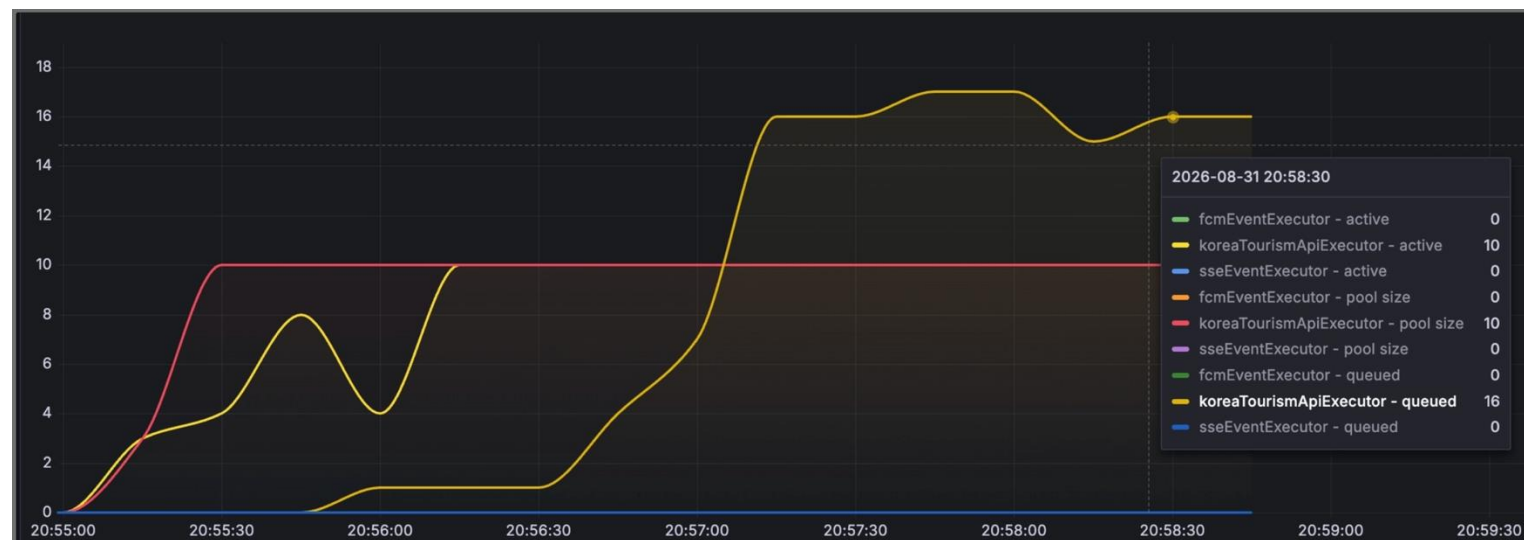
Test 1 결과 분석

톰캣 스레드



톰캣 스레드가 전부 블로킹 → 클라이언트 요청부터 병목

전용 스레드



전용 스레드를 대기 큐에서 대기하는 수가 늘어난다

톰캣 스레드, 전용 스레드 모두 블로킹된다

Test 1 결과 분석

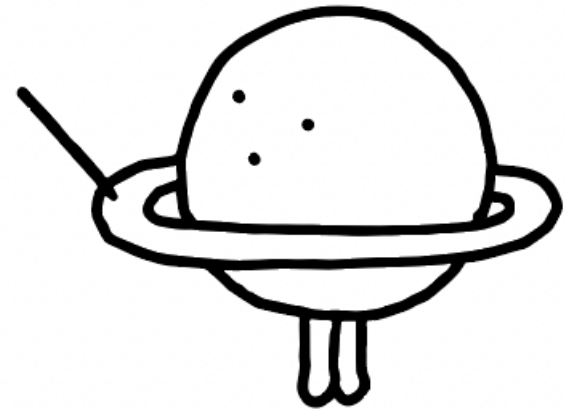
GC, CPU



안정적인 지표

블로킹때문에 자원이 낭비되고 있다

WebClient 전환 (Test2)



RestClient vs WebClient

RestClient

```
String result = restClient.get()  
    .uri("https://api.example.com/tours")  
    .header("Authorization", "Bearer " + token)  
    .retrieve()  
    .body(String.class);
```

WebClient

```
Mono<String> result = webClient.get()  
    .uri("https://api.example.com/tours")  
    .header("Authorization", "Bearer " + token)  
    .retrieve()  
    .bodyToMono(String.class);
```

공통점: 요청 체이닝이 쉽다

차이점: 내부에서 사용하는 네트워크 엔진이 다르다

네트워크 엔진

socket()
빈 소켓(파일 디스크립터) 생성

내 소켓



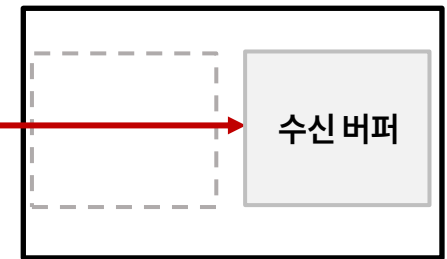
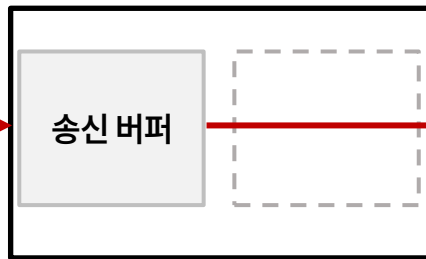
connect()
TCP handshake



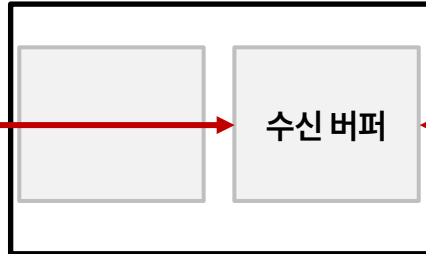
상대 소켓



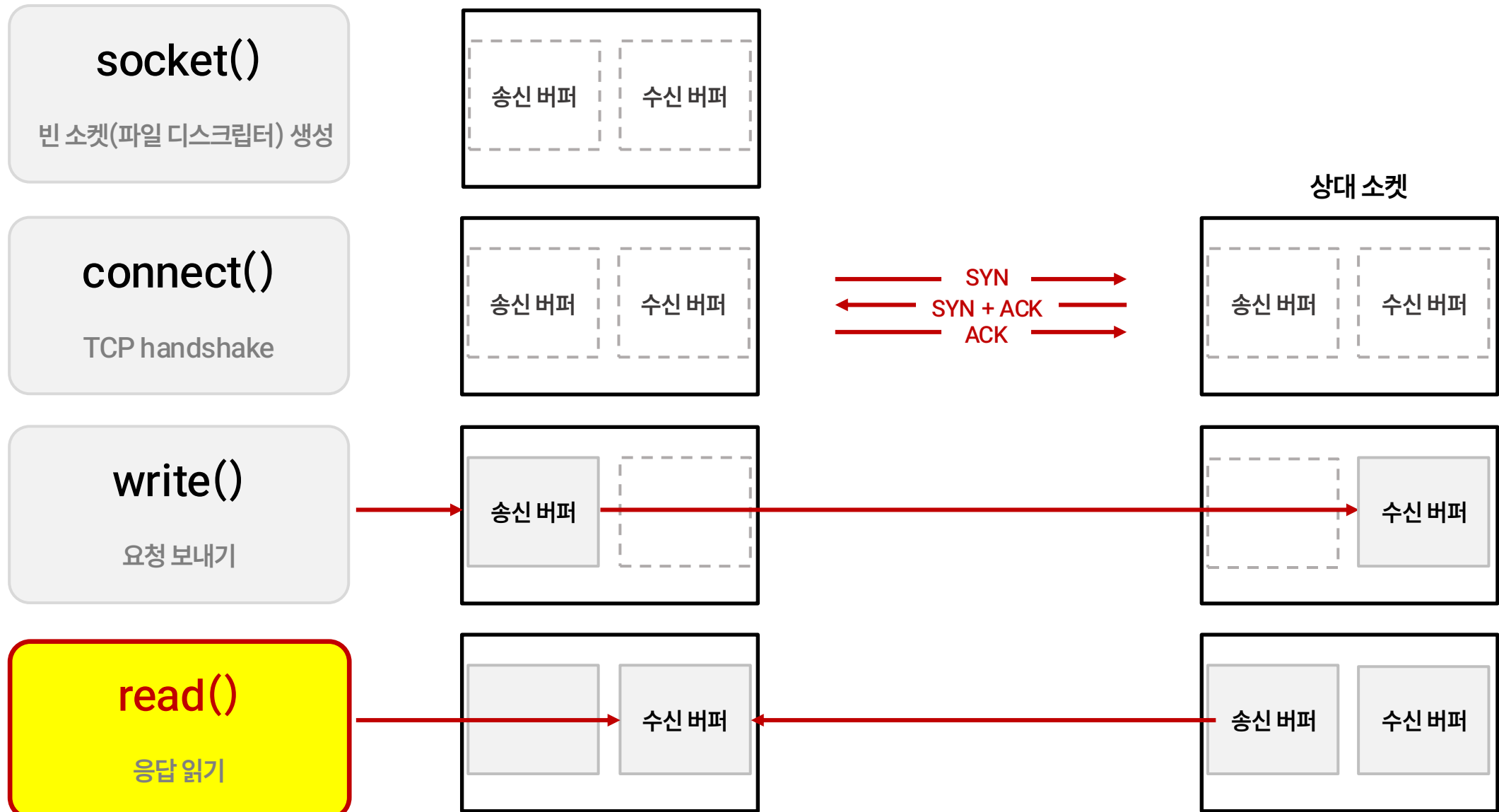
write()
요청 보내기



read()
응답 읽기



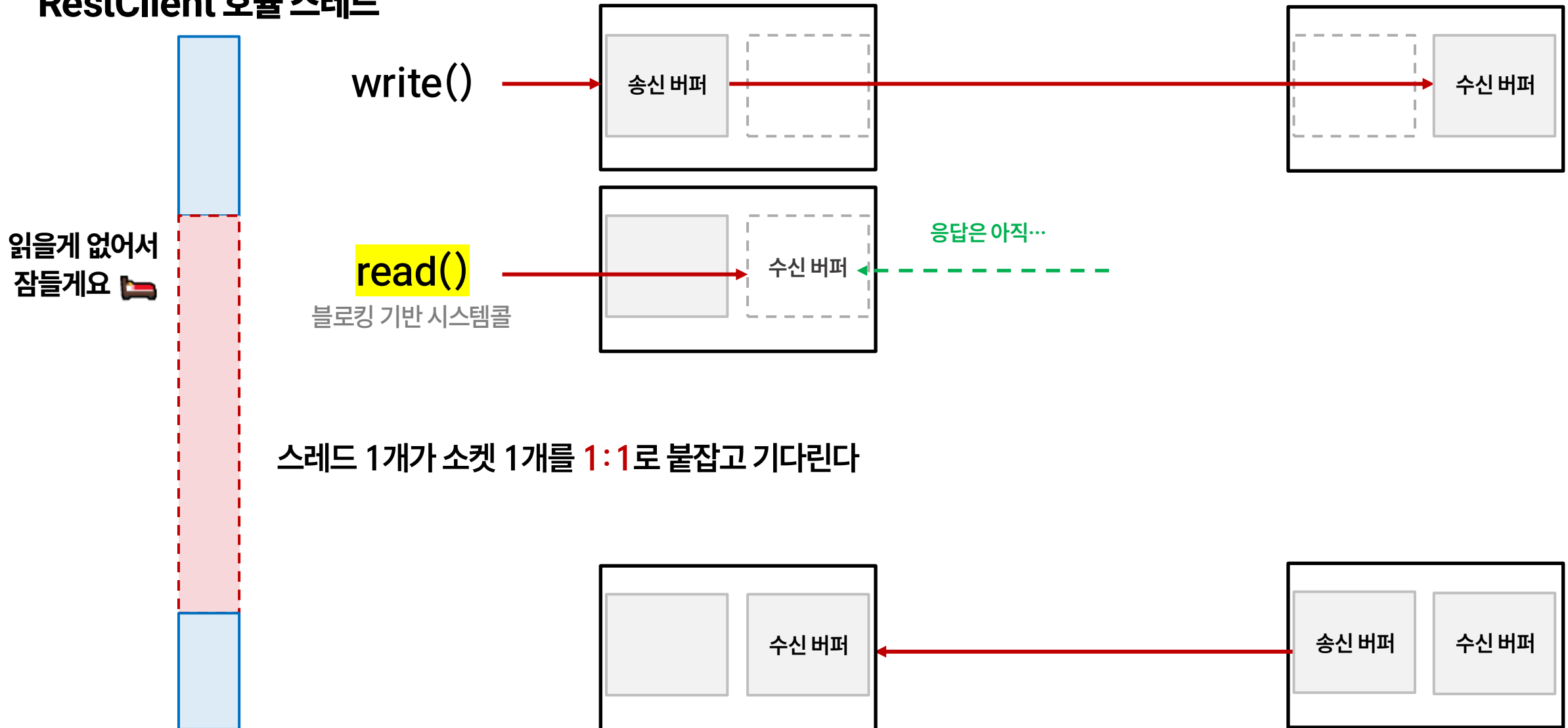
네트워크 엔진 – RestClient와 WebClient의 차이



여기서 스레드가
어떻게 동작하느냐가 다르다

RestClient에서 응답을 기다리는 방법

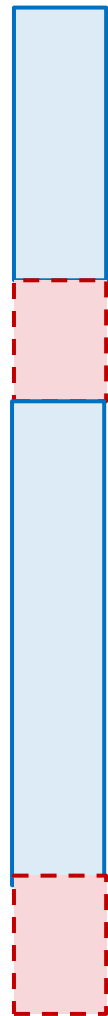
RestClient 호출 스레드





WebClient에서 응답을 기다리는 방법

이벤트 루프 스레드

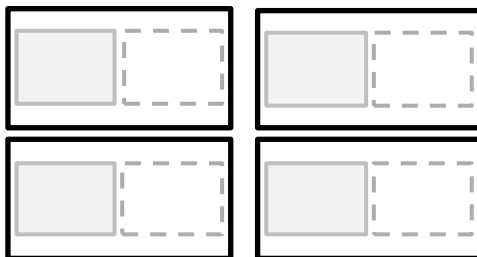


write()



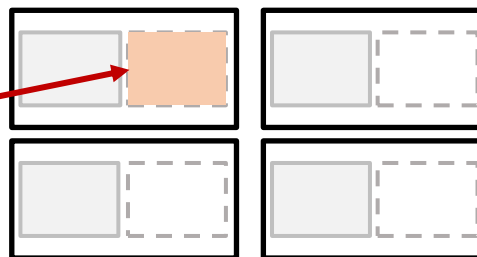
epoll_wait()

소켓들의 상태 변화 감시



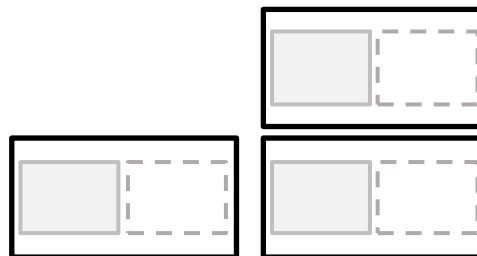
read()

응답 도착하면 그 때
읽기 + 콜백 실행



epoll_wait()

소켓들의 상태 변화 감시



스레드 1개가 소켓 여러개를 담당한다
-> 전체적인 대기 시간 줄어든다.

응답이 도착함을 확인한 후에 read()한다
-> read()에서 블로킹이 일어나지 않는다.

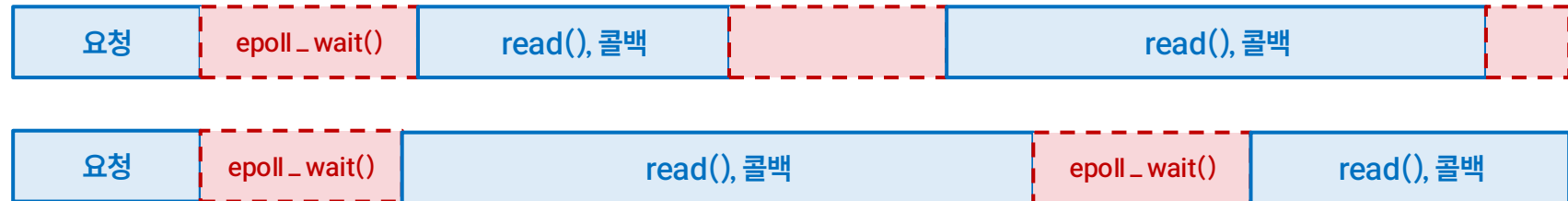
문제 1) 외부 서버를 찌르는 전용 스레드는 블로킹된다

전용 스레드



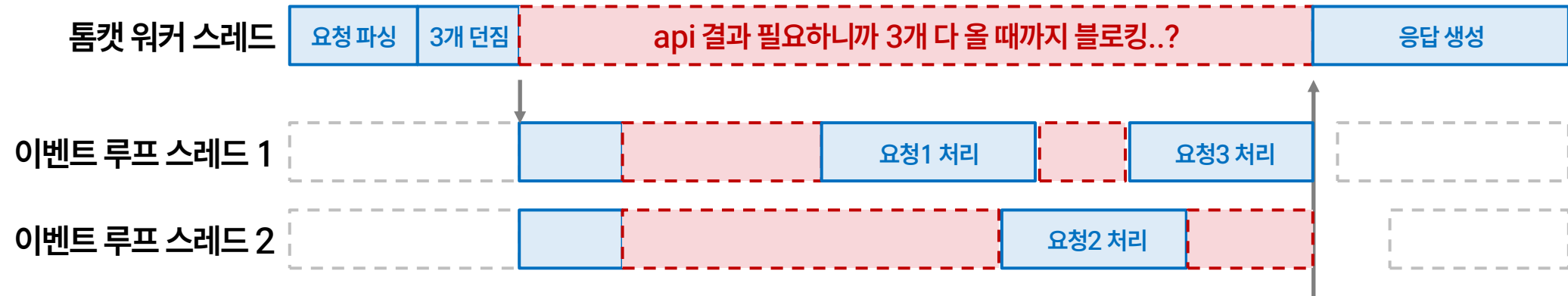
RestClient

이벤트 루프 스레드



WebClient

문제 2) 비동기로 던져도 join()에서 톰캣 스레드가 블로킹된다





콜백 - 응답이 오면 할 일을 미리 등록해둔다

이벤트 루프 스레드



write()



epoll_wait()
소켓들의 상태 변화 감시

read()



응답 도착하면 그 때
읽기 + **콜백 실행**



이거 응답 본문
비어있으면
로그 남겨주시고요

DTO로
변환해주세요



콜백 - *Mono를 통해 등록할 수 있다.

```
return webClient.get() RequestHeadersUriSpec<capture of ?>
    .uri(uri) capture of ?
    .retrieve() ResponseSpec
    .bodyToMono(KoreaTourismRelatedSpotResponse.class) Mono<KoreaTourismRelatedSpotRespons...>
    .switchIfEmpty(Mono.fromRunnable(() ->
        log.warn("한국관광공사 연관 관광지 API 응답 본문이 비어있음: areaCode={}, sigunguCode={},
            areaCode, sigunguCode)))
    .map( KoreaTourismRelatedSpotRespons... response -> {
        if (!response.isSuccess()) {
            log.warn("한국관광공사 연관 관광지 API 응답 실패: resultCode={}, resultMsg=",
                response.getResultCode(), response.getResultMsg());
            return RelatedSpotResult.failure();
        }
        // API 호출 성공, 데이터가 비어있어도 success로 반환
        return RelatedSpotResult.success(response.getRelatedSpots());
    }
```

응답 오면 DTO
변환해주세요

```
private Mono<List<RelatedSpotResult>> getRelatedSpotsByAreaCode(TourApiAreaCode areaCode) {
    // 여러 시군구 코드에 대해 WebClient로 비동기 논블로킹 병렬 호출
    List<Mono<RelatedSpotResult>> monos = areaCode.getSigunguCodes().stream() Stream<Integer>
        .map( Integer sigunguCode -> koreaTourismRelatedSpotClient.searchRelatedSpots(
            areaCode.getAreaCode(),
            sigunguCode
        )) Stream<Mono<...>>
        .toList();

    return Mono.zip(monos, Object[] results -> Arrays.stream(results) Stream<Object>
        .map( Object result -> (RelatedSpotResult) result) Stream<RelatedSpotResult>
        .toList());
}
```

3개 api 결과 하나로
합쳐주세요

*Mono: CompletableFuture와 비슷하게 결과가 아직 없고, 나중에 채워질 자리를 나타내는 객체



콜백 - 톰캣 스레드 블로킹을 완전히 없애지 못한다.

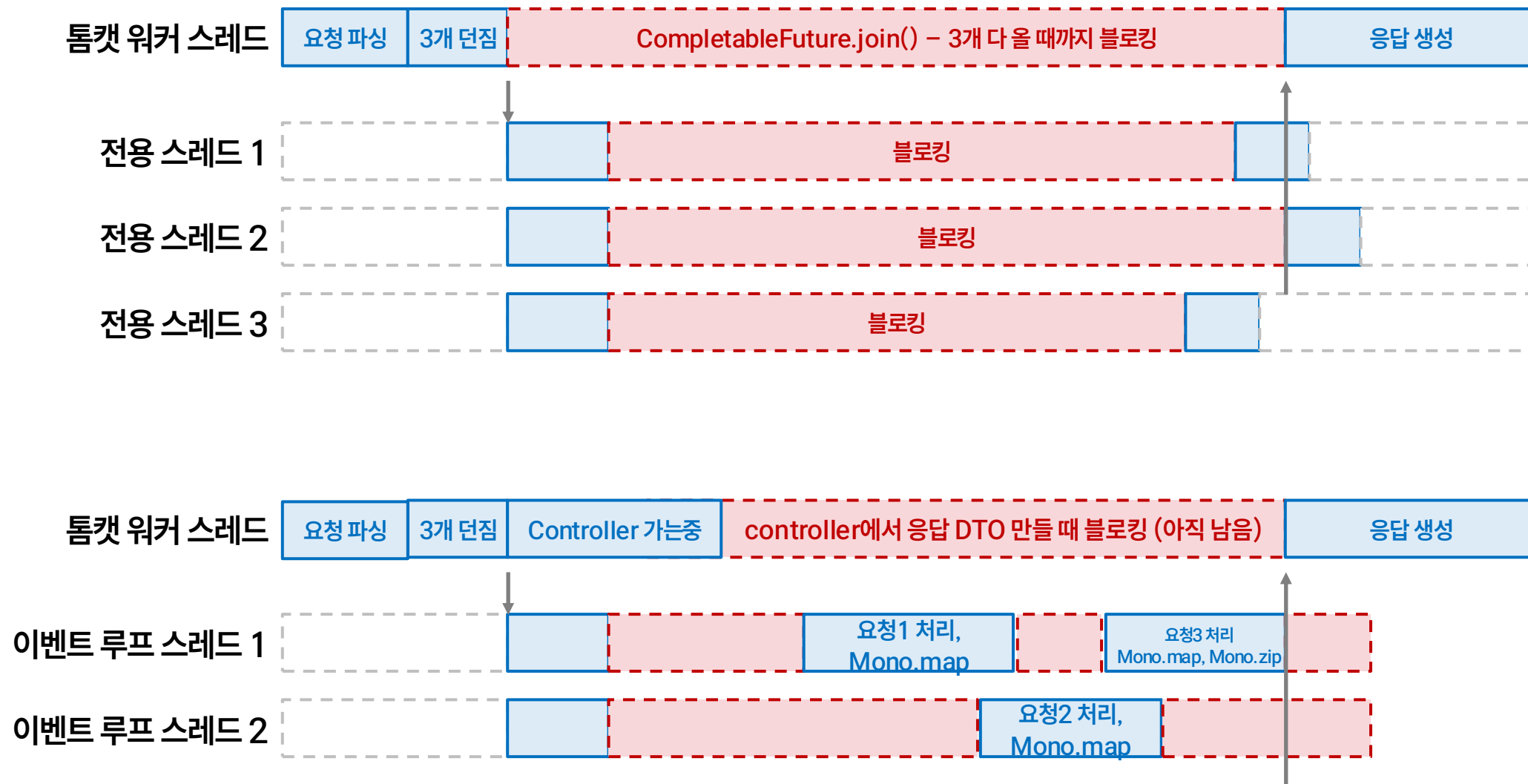
```
@GetMapping("/spots/{id}/related")
public ResponseEntity<List<RelatedSpotResponse>> getRelatedSpots(@PathVariable Long id) {
    List<RelatedSpotResponse> result = relatedSpotService.getRelatedSpots(id)
        .block(); // Mono<List<RelatedSpotResponse>> -> List<RelatedSpotResponse>

    return ResponseEntity.ok(result);
}
```

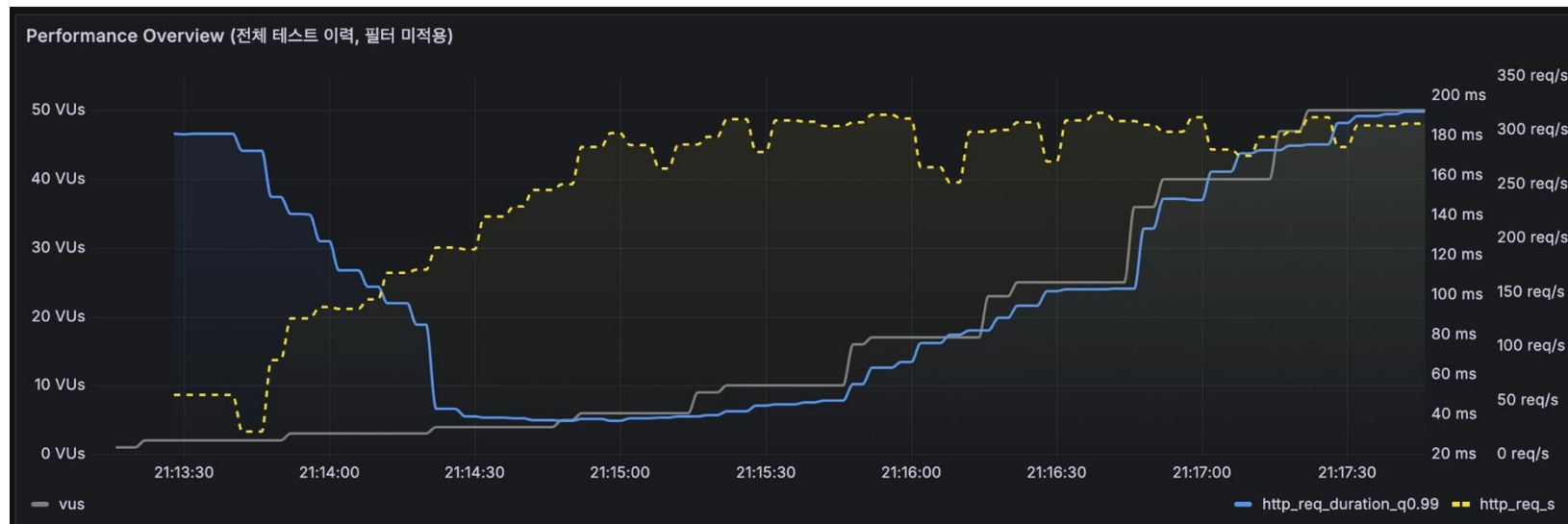
콜백을 통해 블로킹 시점을 최대한 늦출 수 있었다(컨트롤러에서 블로킹)

문제 2 조금 해결

문제 2) 비동기로 던져도 join()에서 톰캣 스레드가 블로킹된다



Test 2 결과 분석



 응답 시간(latency)  처리량(tps)

264

평균 TPS

312

최대 TPS

166 ms

p95 latency

179 ms

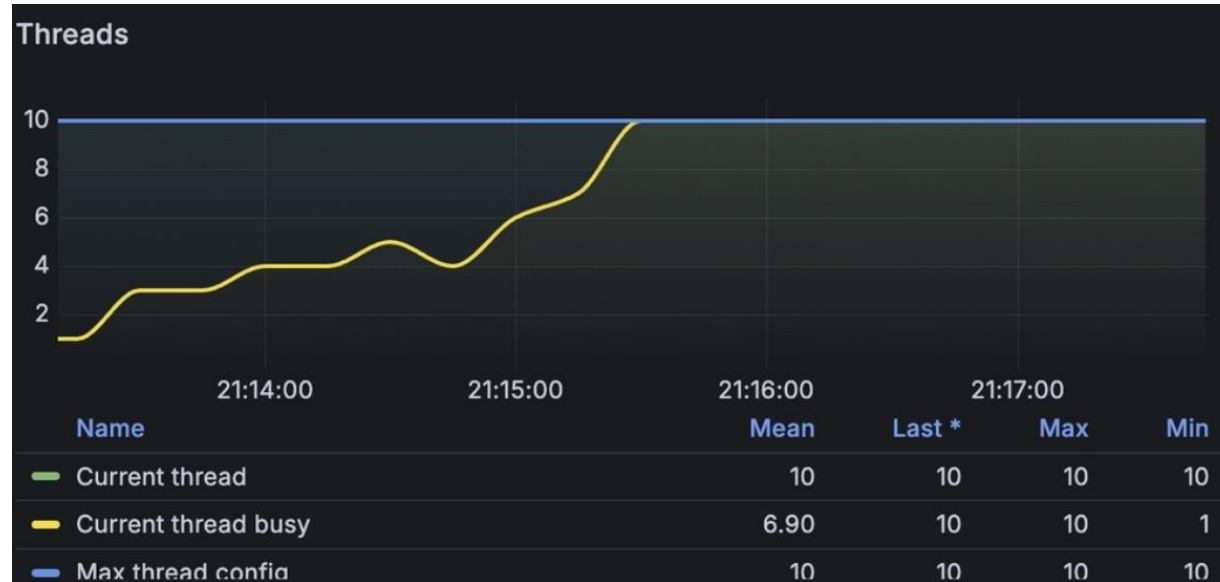
p99 latency

Test 1 대비 p95 13.2s → 166ms
부하가 올라가도 latency가 방어된다

외부 API 호출에서 병목이 존재하지 않으니 응답 시간이 지켜진다

Test 2 결과 분석

톰캣 스레드



톰캣 스레드 병목이 여전히 존재

Test 2 결과 분석

GC, CPU



600

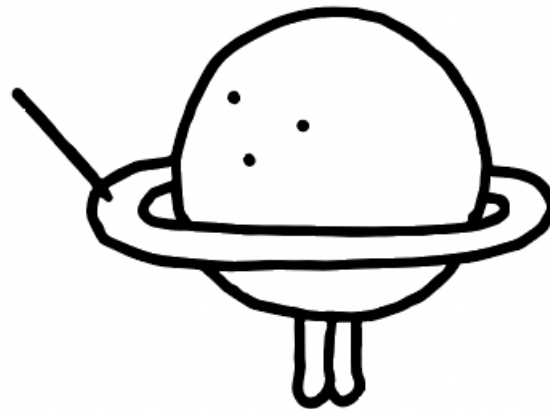
GC count

21.7ms

GC MAX Pause Time

외부 API 블로킹 개선으로 인해 자원 낭비가 줄고, GC 빈도와 STW 시간이 증가했다.

**톰캣 스레드
블로킹 문제 개선하기
(Test3)**

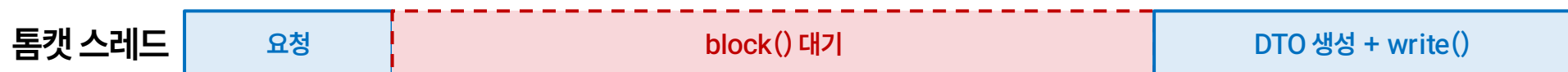




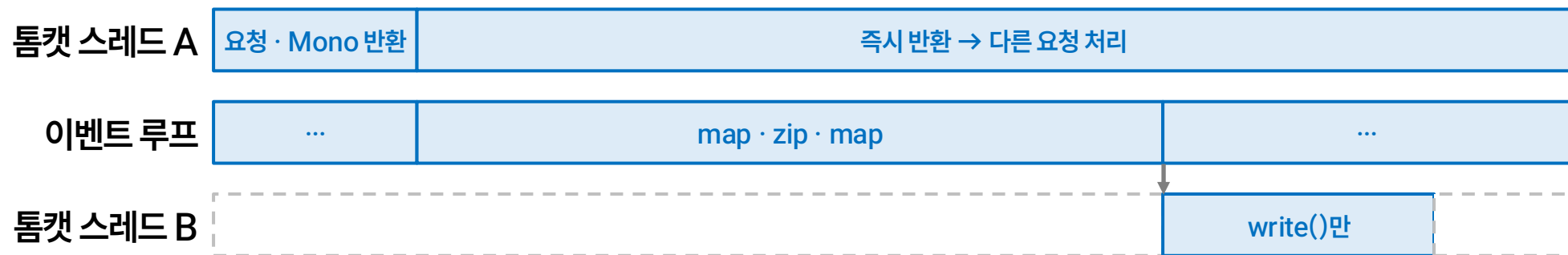
Controller에서 Mono를 return하기

Spring WebFlux*가 아니어도, Mono를 return하면 **톰캣 스레드가 write()를 하지 않고 바로 반환될 수 있다**
문제2(톰캣 스레드 블로킹) 해결!

Test 2 – DTO block



Test 3 – Mono return

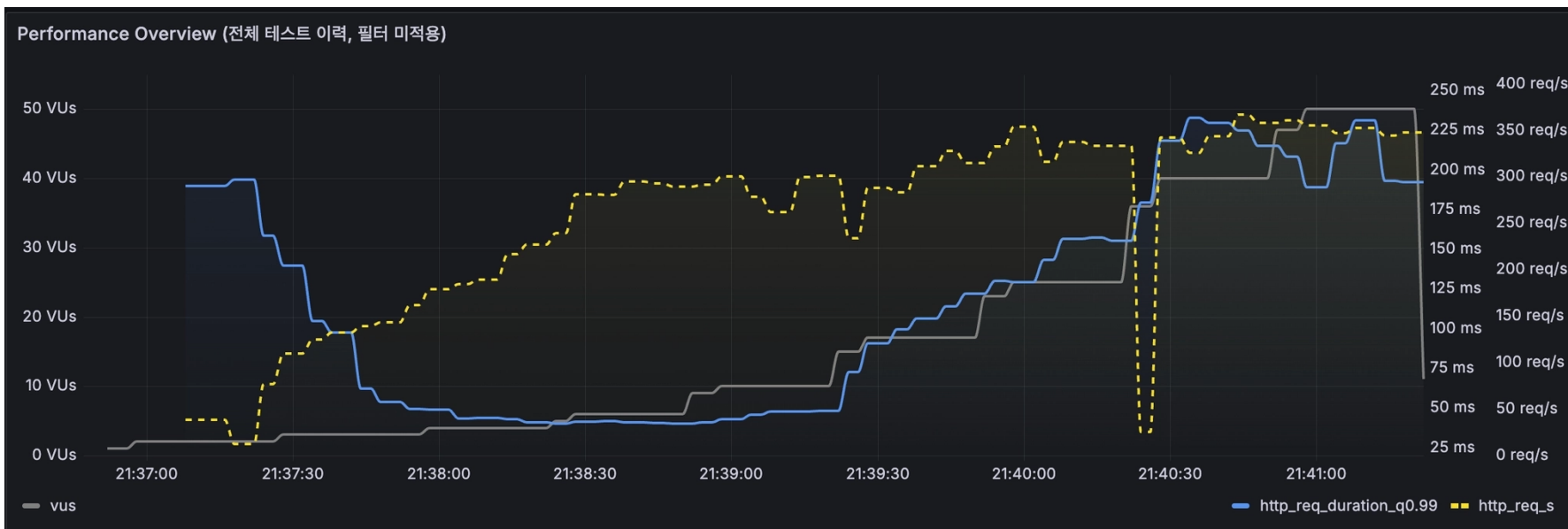


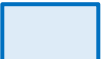
MVC + Mono: 앞단(클라이언트 – 서버)이 톰캣이라 **다른 톰캣 스레드가 잠깐 할당되어 write()만** 해준다

WebFlux + Mono: 앞단(클라이언트 – 서버)도 Netty라 소켓 write()도 이벤트 루프가 처리한다.

*Spring WebFlux: 서블릿/톰캣 기반이 아닌, Reactor Netty 등 논블로킹 이벤트 루프 위에서 동작하는 스프링의 리액티브 웹 프레임워크

Test 3 결과 분석



 응답 시간(latency)  처리량(tps)

297

평균 TPS

355

최대 TPS

146 ms

p95 latency

175 ms

p99 latency

Test 2 대비 평균 tps 264 → 297

톰캣 스레드



톰캣 스레드 병목도 사라짐

Test 3 결과 분석

GC, CPU



610

GC count

34.8ms

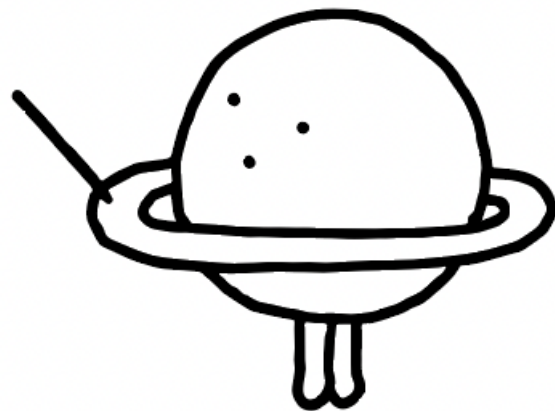
GC MAX Pause Time

Test 2 대비

Max Pause Time 21.7ms → 34.8ms

톰캣 스레드 블로킹 개선으로 인해 자원 낭비가 줄고, GC 빈도와 STW 시간이 증가했다.

결과 정리, 결론



전용 스레드에서의
블로킹 문제 개선툰캣 스레드에서의
블로킹 문제 개선

	Test 1 RestClient	Test 2 WebClient	Test 3 WebClient + Mono return
평균 TPS	3.07	264	297
최대 TPS	3.33	312	355
p95 latency	13,200 ms	166 ms	146 ms
p99 latency	16,800 ms	179 ms	175 ms

Test 1 → Test 2

최대 TPS 약 94배 증가

Test 1 → Test 2

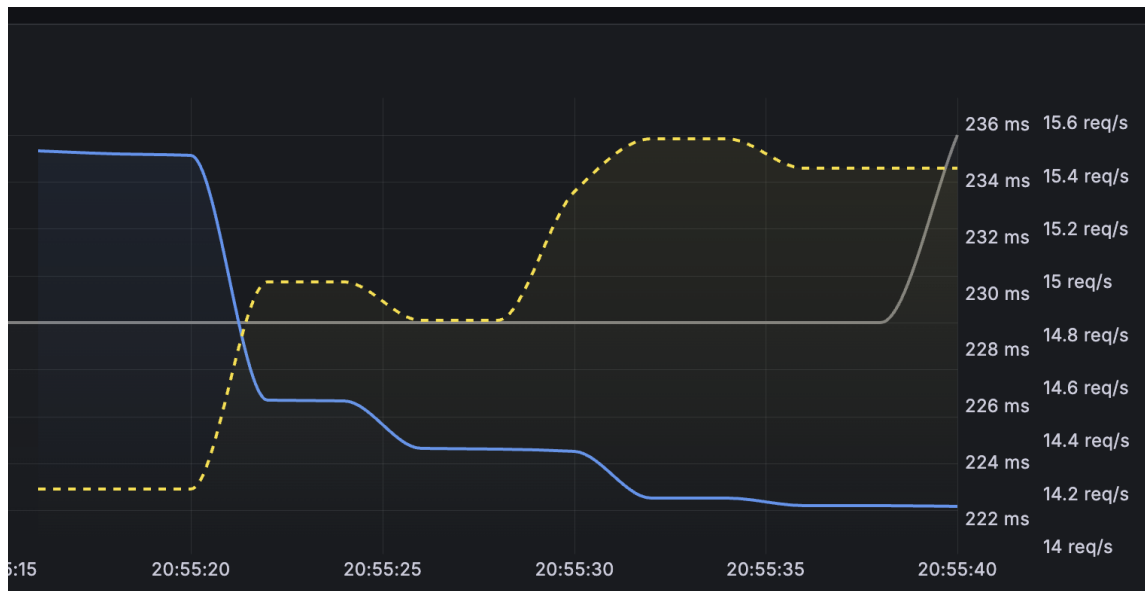
p95 latency 98.7% 감소

참고) RestClient의 전용 스레드풀을 50개로 늘려도(Test 4) WebClient만큼 따라오지 못했다

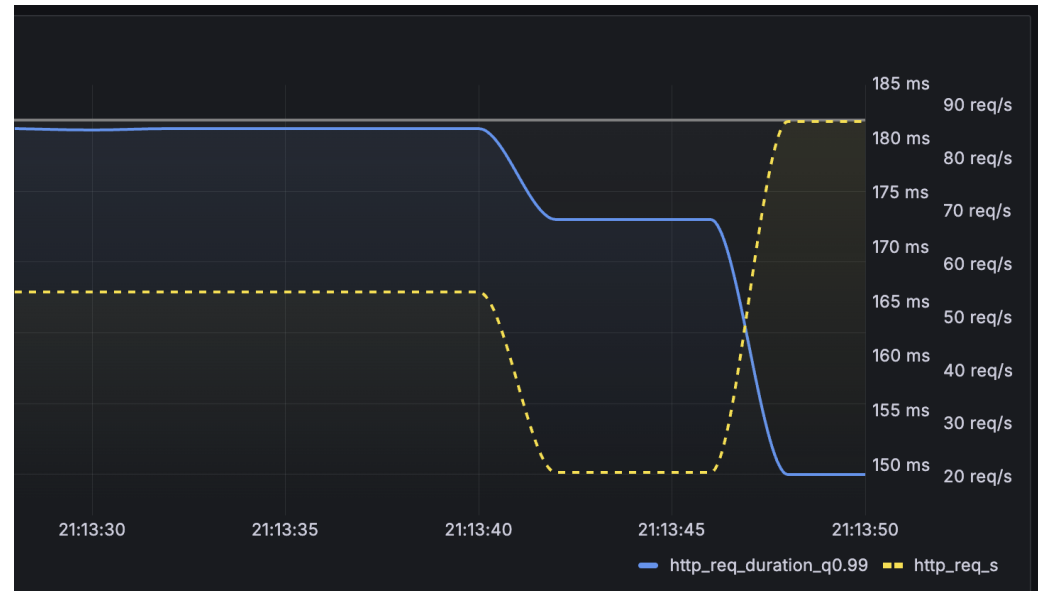
큰 개선은 RestClient → WebClient 전환에서 나온다

가설 1

저부하: WebClient로 바뀌도 latency는 별 차이 없다
고부하(동시 요청 > 전용 스레드풀 10개): WebClient가 더 낮고 안정적이다



Test1 저부하 구간



Test2 저부하 구간

가설 2

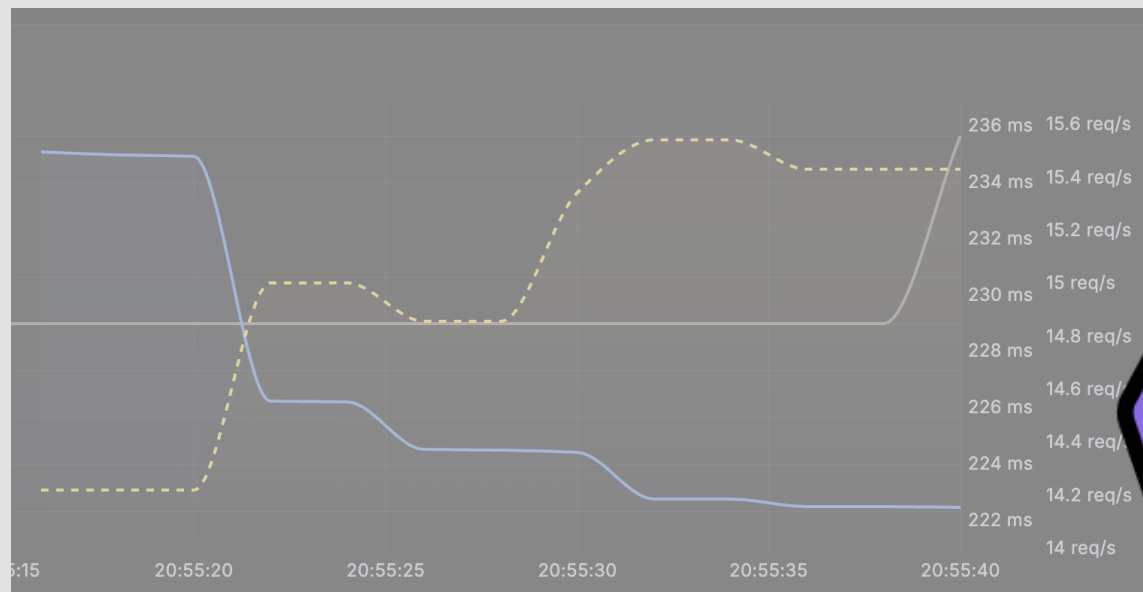
WebClient로 전환하면 처리량(TPS)이 늘어난다

가설 3

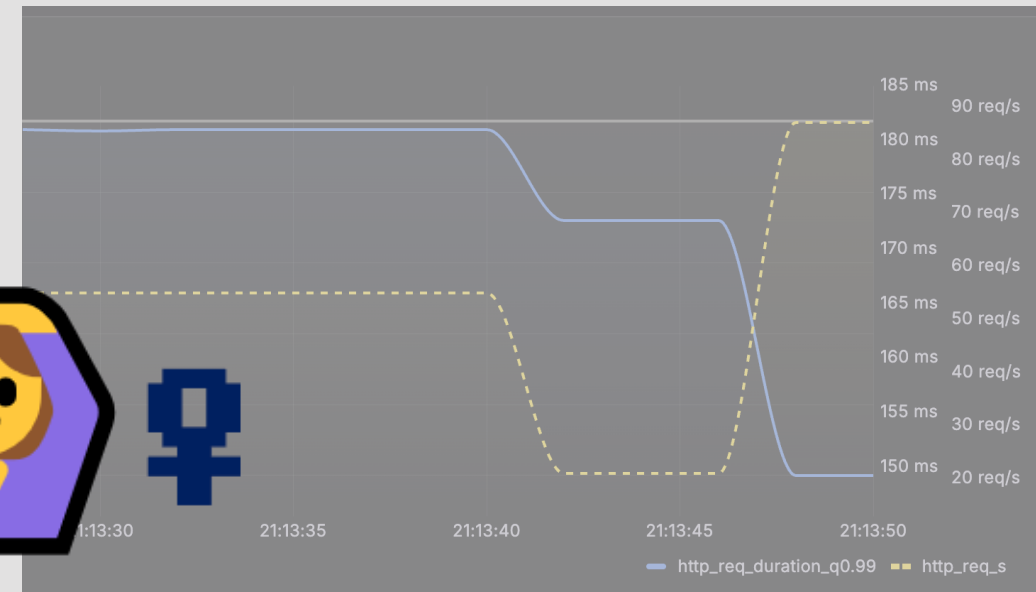
Controller에서 Mono를 return하면 처리량이 더 늘어난다

가설 1

저부하: WebClient로 바뀌도 latency는 별 차이 없다
고부하(동시 요청 > 전용 스레드풀 10개): WebClient가 더 낮고 안정적이다



Test1 저부하 구간



Test2 저부하 구간

가설 2

WebClient로 전환하면 처리량(TPS)이 늘어난다

가설 3

Controller에서 Mono를 return하면 처리량이 더 늘어난다



1

RestClient → WebClient는 무조건 바꾼다

약 94배 개선. 전용 스레드풀을 50개로 늘려도(Test 4) 따라오지 못했다

2

Controller Mono return은 필수는 아니다

약 12% 개선이라, 코드 복잡도와 비일관성을 감수할 만큼은 아닐 수 있다

3

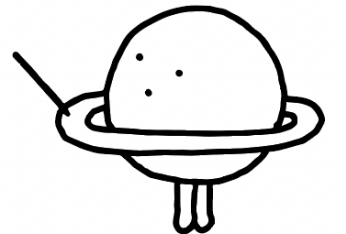
그럼에도 튜립은 Mono return까지 적용한다

외부 API 때문에 톰캣 스레드가 묶이면 이 기능뿐 아니라 다른 기능의 처리량까지 떨어지기 때문

WebClient + Controller Mono return을 적용한다

**리액티브 프로그래밍?
논블로킹/비동기 프로그래밍?
선언적 프로그래밍?**

**찍먹해봤는데 어려웠다..
내 사고는 너무 동기적으로 되어 있다**



Q&A

